

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Работа с сетью

Модуль `socket`

Немного теории:

Сокет в веб - это технология, позволяющая создавать интерактивное соединение для обмена сообщениями в онлайн-режиме. Соединение создаётся в сети между сервером и клиентом (браузером).

До того, как клиент и сервер начнут обмениваться сообщениями, они должны установить соединение. Это делается установкой "рукопожатия", где клиент и сервер посылают запрос на соединение, и, если сервер хочет этого, то он пошлет ответ, подтверждающий соединение.

Если вы наберёте [Codeby.net](https://codeby.net) в своем веб-браузере, он откроет сокет и подключится к Codeby.net, чтобы получить веб-страницу и показать ее вам в формате HTML.

Рассмотрим пример:

Сначала мы импортируем `socket` библиотеку. Затем мы создаем сокет с помощью `socket.socket` функции. Мы передали `AF_INET` как семейство адресов IPv4, и `SOCK_STREAM` означает, что мы использовали протокол `TCP`.

После создания сокета мы теперь можем подключиться к серверу через определенный номер порта. Нам нужно знать IP-адрес и номер порта для подключения. Мы будем использовать порт 80, который является http-портом по умолчанию и `yandex.ru` в качестве примера.

Перед подключением к удаленному хосту выясним его IP-адрес. В Python получение IP-адреса довольно просто. Используем `gethostbyname` функцию, чтобы получить IP-адрес хоста.

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = 'yandex.ru'
remote_ip = socket.gethostbyname(host)
print(f'IP Address of '{host}' is '{remote_ip}')
```

my_req ×

IP Address of yandex.ru is 77.88.55.55

Если нам нужно подключиться к определённому порту, нужно его указать. Подключение к серверу осуществляется с помощью метода `connect`.

Обратите внимание, что IP-адрес у Яндекса может измениться при следующем запросе. Дело в том, что у таких гигантов есть целая сеть серверов, и при обращении через имя хоста, нас могут подключить к любому из них.

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = 'yandex.ru'
port = 80
remote_ip = socket.gethostbyname(host)
s.connect((remote_ip, port))
print(f"{'IP Address of '}{host}{' is '}{remote_ip}")
print(f"{'Socket connected to '}{host}{' on ip '}{remote_ip}")
```

my_req2 ×

```
IP Address of yandex.ru is 77.88.55.55
Socket connected to yandex.ru on ip 77.88.55.55
```

Если нам нужно соединиться с конкретным сервером, то вместо имени хоста укажем нужный IP-адрес.

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = '77.88.55.70'
port = 80
s.connect((host, port))
print(f"{'Socket connected to '}{host}")
```

my_req3 ×

```
Socket connected to 77.88.55.70
```

При работе с сетевыми соединениями конечно нужно перехватывать исключения для более стабильной работы программы. Напишем порт 82, который наверняка не используется, и добавим обработку исключений.

Если какая-либо из функций сокета завершается с ошибкой, то python генерирует исключение с именем `socket.error`, которое должно быть перехвачено.

Если по каким-то причинам сокет не создавался, то дальнейшие действия не имеют смысла. Поэтому, используя модуль `sys`, в случае возникновения ошибки, мы просто выходим из программы.

Чтобы гарантированно закрыть любое соединение, в конце добавим `close()` в блоке `finally`.

Также мы можем установить тайм-аут соединения `settimeout(5)` указав в секундах время ожидания. Если этого не сделать, то будет время ожидания в зависимости от тайм-аута установленного операционной системой.

```

import socket
import sys

try:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(5)
except socket.error:
    print('Failed to create socket!')
    sys.exit()
host = '77.88.55.70'
port = 82
try:
    s.connect((host, port))
    print(f"{'Socket connected to '}{host}")
except Exception as e:
    print(f"{'Failed connected: '}{e}")
finally:
    s.close()

```

my_req4 x

Failed connected: timed out

Socketserver протокол TCP

Сетевой протокол — набор правил и действий (очередности действий), позволяющий осуществлять соединение и обмен данными между двумя и более включёнными в сеть устройствами.

TCP - «гарантированный» протокол с предварительным установлением соединения («рукопожатием»).

Для понимания взаимодействия клиент-сервер, рассмотрим чат в локальной сети. То есть клиент с сервером будут обмениваться сообщениями. Важный момент – первым запускается сервер, а потом уже клиент коннектится к серверу.

По-умолчанию адрес локального сервера **127.0.0.1**. Для того чтобы были доступны все IPv4-адреса, можно указать **0.0.0.0** в качестве IP-адреса.

Метод **bind()** принимает 2 аргумента – адрес сервера и порт, и связывает сокет с данным хостом и портом.

С помощью метода **listen()** мы запустим для данного сокета режим прослушивания. Метод принимает один аргумент - максимальное количество подключений в очереди. И если там будет стоять 1, то подключиться к серверу сможет только 1 клиент.

С помощью метода **accept()** мы принимаем подключение, метод возвращает кортеж с двумя элементами - новый сокет и адрес клиента.

Функция `recv()` может быть использована для приема данных от TCP и UDP сокетов. В скобках указывается размер буфера - количество байт для чтения, так 1024 байт = 1 кб.

Функция `send()` отправляет данные. Функции `recv()` и `send()` работают с типом `bytes`, который требует преобразования из/в строку при помощи `encode()/decode()` соответственно.

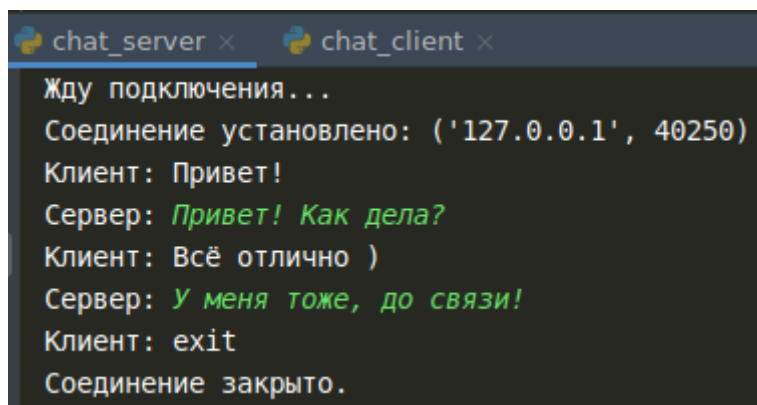
Код сервера:

```
import socket

s = socket.socket()
print("Жду подключения... ")
s.bind(('0.0.0.0', 4444))
s.listen(1)
conn, addr = s.accept()
try:
    print("Соединение установлено:", addr)
    while True:
        client_mes = conn.recv(1024).decode()
        if not client_mes:
            break
        print("Клиент:", client_mes)
        if client_mes == "exit":
            break
        conn.send(input("Сервер: ").encode())
    print("Соединение закрыто.")
except Exception as e:
    print("Ошибка: ", e)
finally:
    conn.close()
```

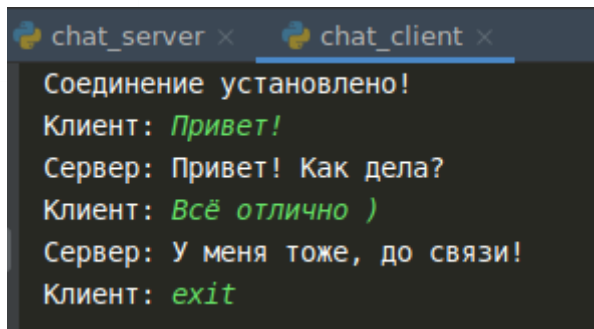
Для клиента не требуется привязывать сокет к порту с помощью `bind()`, так как ядро операционной системы само позаботится о назначении источника IP и назначает временный номер порта.

На скриншоте примера работы чата ниже видно, что переданный серверу номер порта клиента отличается от установленного.



```
chat_server x chat_client x
Жду подключения...
Соединение установлено: ('127.0.0.1', 40250)
Клиент: Привет!
Сервер: Привет! Как дела?
Клиент: Всё отлично )
Сервер: У меня тоже, до связи!
Клиент: exit
Соединение закрыто.
```

Пример работы чата клиентской части.



```
chat_server x chat_client x
Соединение установлено!
Клиент: Привет!
Сервер: Привет! Как дела?
Клиент: Всё отлично )
Сервер: У меня тоже, до связи!
Клиент: exit
```

Код клиента:

```
import socket

s = socket.socket()

try:
    s.connect(('127.0.0.1', 4444))
    print("Соединение установлено!")
    while True:
        client_mes = input("Клиент: ")
        s.send(client_mes.encode())
        if client_mes == "exit":
            break
        server_mes = s.recv(1024).decode()
        print("Сервер:", server_mes)
except Exception as e:
    print("Ошибка: ", e)
finally:
    s.close()
```

Socketserver протокол UDP

UDP (User Datagram Protocol) — позволяет посылать сообщения (так называемые датаграммы) другим хостам по IP-сети без необходимости предварительного сообщения для установки каналов передачи данных. То есть UDP использует простую модель передачи, без «рукопожатий».

При создании сокета UDP пишем `SOCK_DGRAM`

Функция `recvfrom()` - получает UDP сообщения.

Функция `sendto()` – отправляет UDP сообщения.

Ниже простейший пример UDP сервера и клиента. В данном коде, сообщения будет отправлять только клиент, а сервер их принимать. Сообщения отправляются в бесконечном цикле, и выход осуществляется нажатием ENTER на стороне клиента.

Код сервера:

```
import socket

udp_port = 2323
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(('', udp_port))
print("Жду сообщение...")
while True:
    msg, addr = sock.recvfrom(1024)
    if not msg:
        break
    print(msg.decode())

sock.close()
```

udp_server2 x udp2 x

Жду сообщение...
Привет сервер!
Проверка связи...
Пока!

Код клиента:

```
import socket
import sys

udp_port = 2323
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.connect(('localhost', udp_port))

while True:
    data = s.sendto(input('Введите сообщение: ').encode(), ('', udp_port))
    if not data:
        sys.exit()
```

udp_server2 x udp2 x

Введите сообщение: *Привет сервер!*
Введите сообщение: *Проверка связи...*
Введите сообщение: *Пока!*
Введите сообщение:

Работа с FTP

FTP (File Transfer Protocol). Стандартный протокол, предназначенный для передачи файлов по TCP-сетям. Чтобы сделать ftp-сервер на локальной машине, установите vsftpd

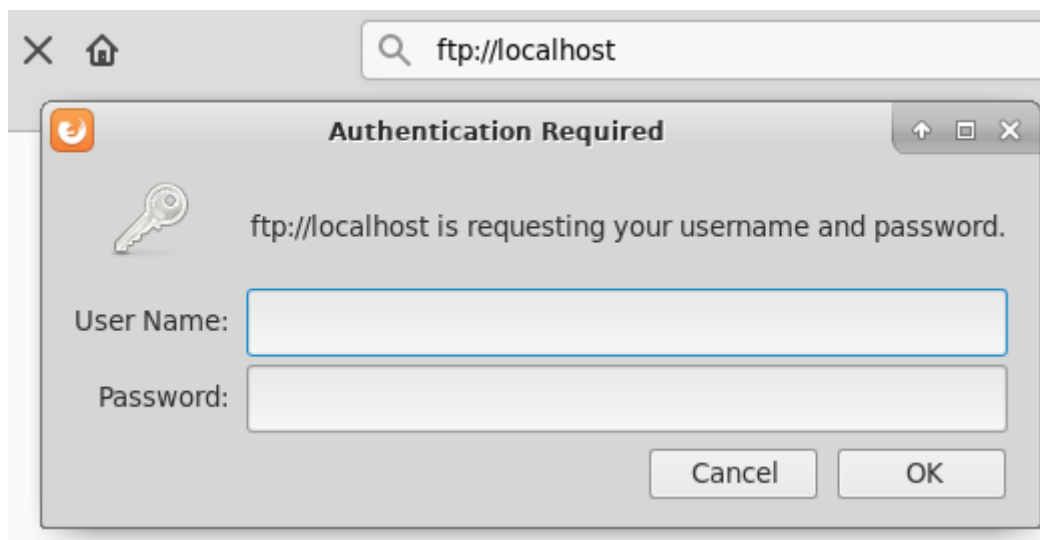
в систему командой **apt install vsftpd**. Затем отредактируйте файл **/etc/vsftpd.conf** в котором нужно раскомментировать строку **#write_enable=YES**. Эта строка разрешает полный доступ (чтение/запись) по FTP. Также раскомментируем строки **#chroot_local_user=YES** и **allow_writeable_chroot=YES**. Эти строки меняют директорию на домашнюю, и разрешают запись в неё. Если какая-то из этих строк отсутствует, то добавить.

Добавьте нового пользователя

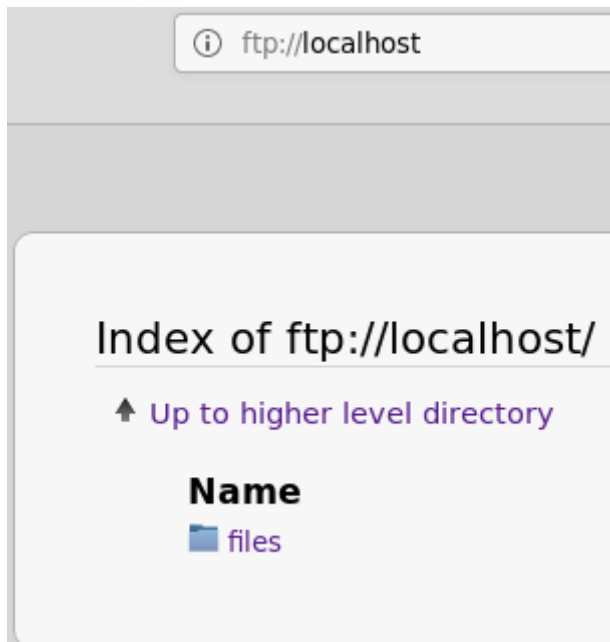
```
root@exp:~# adduser ftpuser
Добавляется пользователь «ftpuser» ...
Добавляется новая группа «ftpuser» (1001) ...
Добавляется новый пользователь «ftpuser» (1001) в группу «ftpuser» ...
Создаётся домашний каталог «/home/ftpuser» ...
Копирование файлов из «/etc/skel» ...
Новый пароль :
Повторите ввод нового пароля :
passwd: пароль успешно обновлён
Изменение информации о пользователе ftpuser
Введите новое значение или нажмите ENTER для выбора значения по умолчанию
    Полное имя []:
    Номер комнаты []:
    Рабочий телефон []:
    Домашний телефон []:
    Другое []:
Данная информация корректна? [Y/n] y
```

В домашней директории созданного пользователя создайте папку files командой **mkdir /home/ftpuser/files**, и перезагружаем **service vsftpd restart**. Теперь можно зайти на ftp через строку браузера <ftp://localhost/>

После этого система попросит ввести логин и пароль локального пользователя, и затем появится его домашняя директория со всем содержимым (войти можно будет только в нее).



Вводим логин и пароль нашего пользователя и получает доступ к директории.



Теперь создадим скрипт для просмотра FTP. Метод `retrlines` с параметром `LIST` показывает содержимое. Для успешного соединения, достаточно указать хост, логин и пароль. Если же вы сменили порт на нестандартный, то нужно будет добавить пару строк

`PORT = 4444`

`ftp.connect(HOST, PORT)`

```
from ftplib import FTP

ftp = FTP("localhost")
ftp.login("ftpuser", "12345")
data = ftp.retrlines('LIST')
print(data)
```

Там у нас только совершенно пустая директория `files`. Теперь загрузим какой-нибудь файл через FTP, например, картинку. Чтобы файл загрузился не в корень, в директорию `files`, которую мы создавали, нужно сменить путь по-умолчанию с помощью метода `cwd()`.

Модуль поддерживает менеджера контекста, поэтому будем им пользоваться. Метод `storbinary()` позволяет отправлять файлы в двоичном режиме передачи с параметром `STOR` и указанием названия файла на выходе. Для выхода используем команду `quit()`.

```
from ftplib import FTP

ftp = FTP("localhost")
ftp.login("ftpuser", "12345")
# Меняем директорию
ftp.cwd('files')
filename = "/var/www/html/images/buratino.jpeg"

with open(filename, 'rb') as fp:
    ftp.storbinary('STOR buratino.jpeg', fp)
ftp.quit()
```

Теперь убедимся, что картинка успешно ушла на наш сервер.

```
from ftplib import FTP

ftp = FTP("localhost")
ftp.login("ftpuser", "12345")
# Меняем директорию
ftp.cwd('files')
data = ftp.retrlines('LIST')
print(data)
```

ftpuser x

```
-rw-----  1 1001    1001      70892 Mar 02 01:01 buratino.jpeg
226 Directory send OK.
```

Также можно передать файл состоящий из строк в режиме передачи ASCII, методом `storlines()`.

```
from ftplib import FTP

ftp = FTP("localhost")
ftp.login("ftpuser", "12345")
# Меняем директорию
ftp.cwd('files')
filename = "/var/www/html/shell.php"

with open(filename, 'rb') as fp:
    ftp.storlines('STOR shell.php', fp)
ftp.quit()
```

Обратное действие – скачать файл с сервера осуществляется методом `retrbinary()` с параметром `RETR` для бинарных файлов и методом `retrlines()` для текстовых. **Обратите внимание** – при загрузке используется режим 'rb', а при скачивании 'wb'.

```
from ftplib import FTP

ftp = FTP("localhost")
ftp.login("ftpuser", "12345")
# Меняем директорию
ftp.cwd('files')
filename = "/var/www/html/buratino.jpeg"

with open(filename, 'wb') as fp:
    ftp.retrbinary('RETR buratino.jpeg', fp.write)
ftp.quit()
```